

# Neural Operators

## PDE-Based Modeling A New Paradigm for Learning Solution Maps

Ali Haghighi

Feb 13, 2026



Université

de Strasbourg

# Outline

01

**Current State**

02

**Neural Operator  
Intuition**

03

**Implementation  
in Our Problem**

04

**Research Gap  
& Next Steps**

# 01

## Current State

*Review of what we've done  
and where we are*

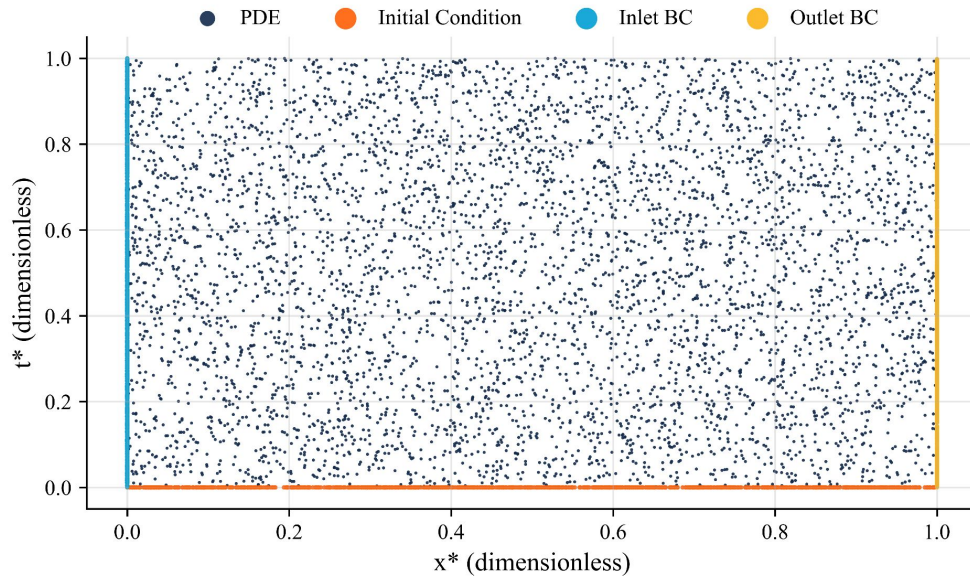
Homogeneous 1D advection-diffusion problem

PINN → Parametric PINN

## Baseline PINN:

a small fully connected MLP:  $2 \rightarrow 12 \rightarrow 12 \rightarrow 12 \rightarrow 1$ , with Tanh in hidden layers and Sigmoid on the output, trained via physics-informed loss.

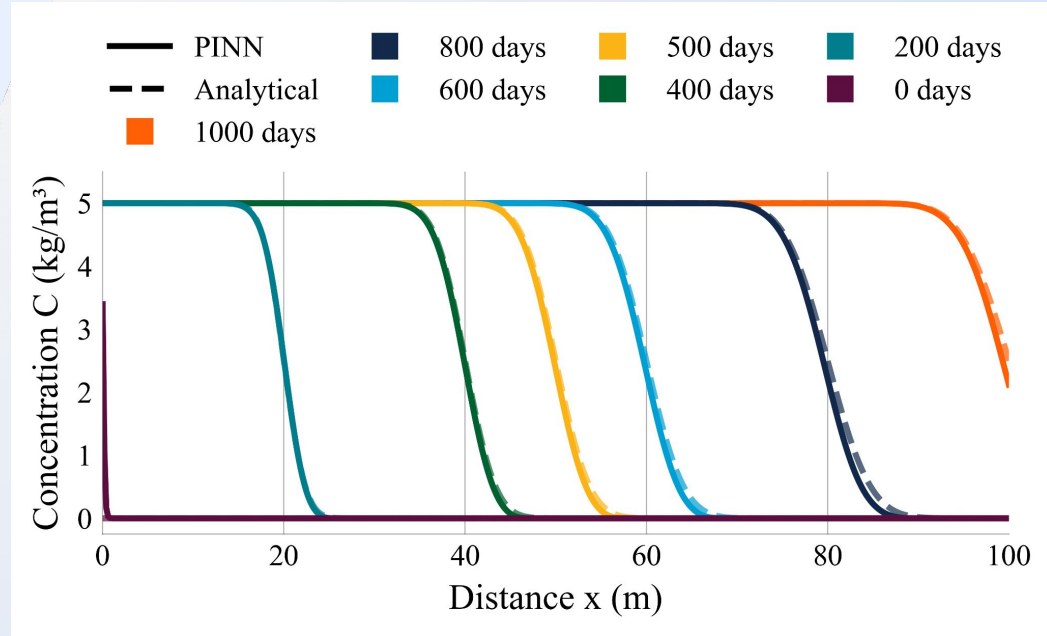
$$(x^*, t^*) \mapsto C^*$$



### Baseline PINN:

a small fully connected MLP:  $2 \rightarrow 12 \rightarrow 12 \rightarrow 12 \rightarrow 1$ , with Tanh in hidden layers and Sigmoid on the output, trained via physics-informed loss.

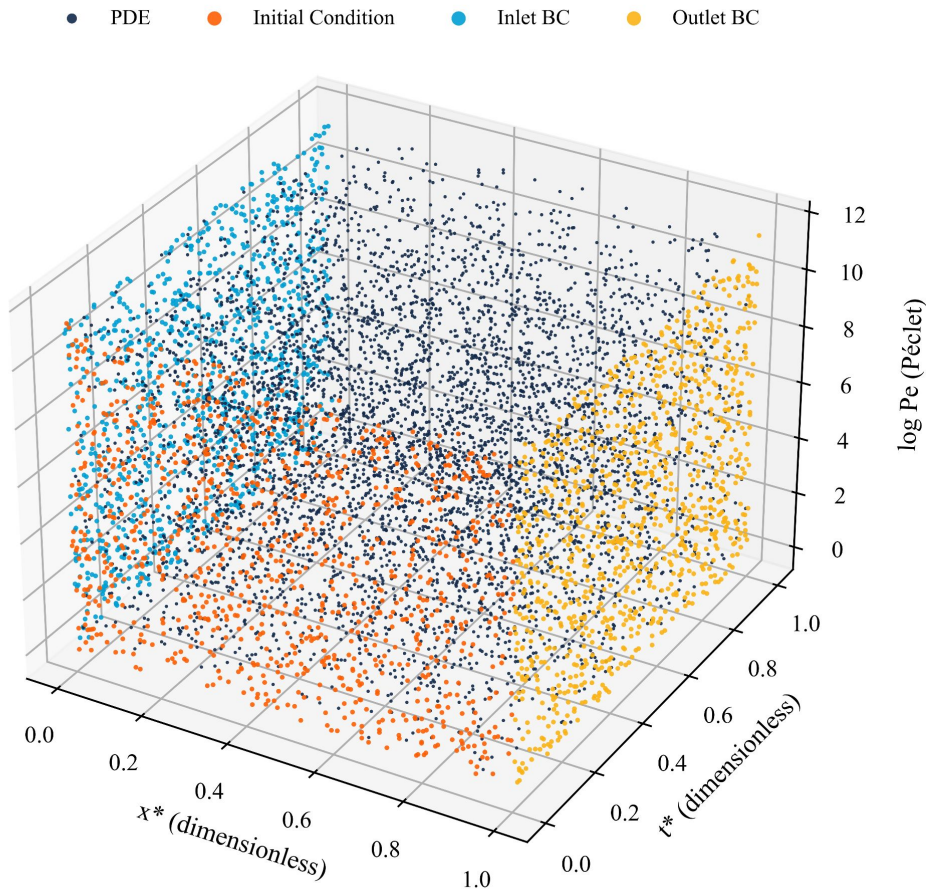
$$(x^*, t^*) \mapsto C^*$$



### Parametric PINN:

A small fully connected MLP:  $3 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 1$ , with Tanh in hidden layers and Sigmoid on the output, trained via physics-informed loss.

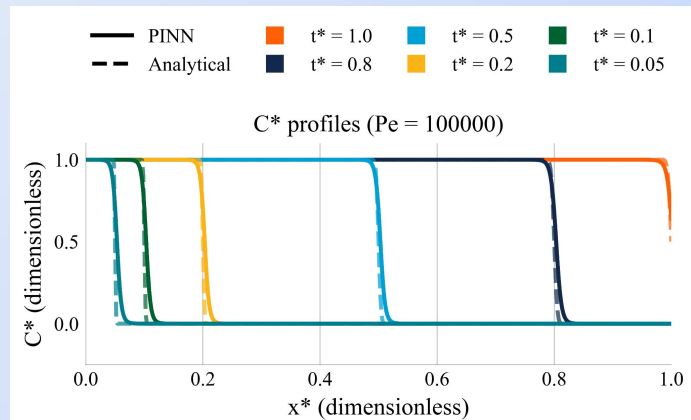
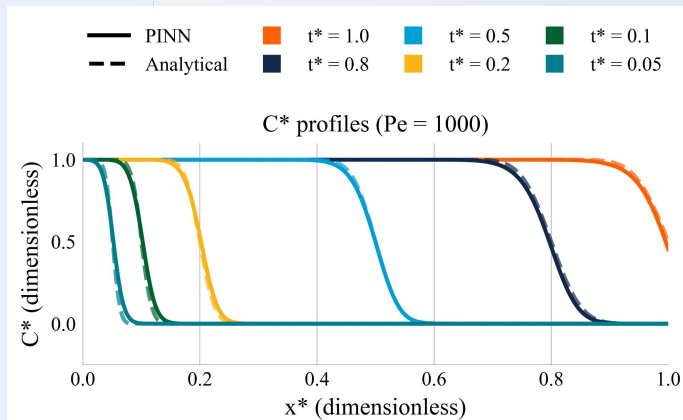
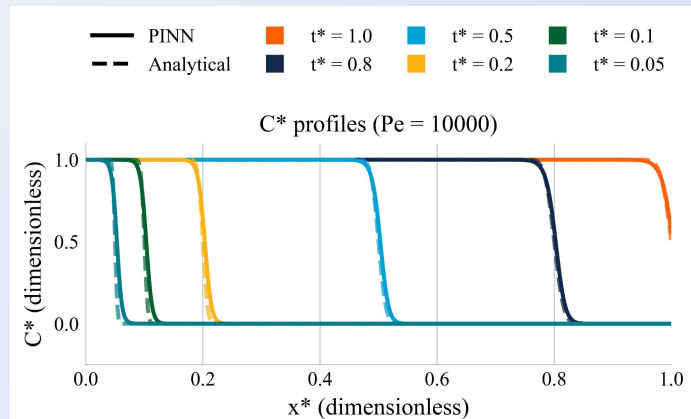
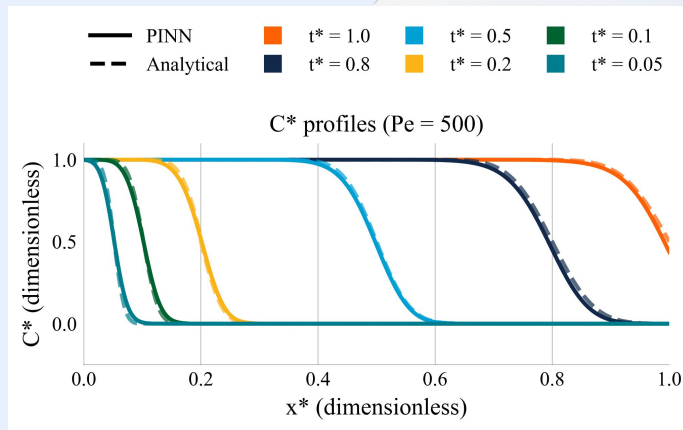
$$(x^*, t^*, \log Pe) \mapsto C^*$$



## Parametric PINN:

A small fully connected MLP:  $3 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 1$ , with Tanh in hidden layers and Sigmoid on the output, trained via physics-informed loss.

$$(x^*, t^*, \log Pe) \mapsto C^*$$



02

# Neural Operator Intuition

*Why neural operators?*  
***Solve vs learn***



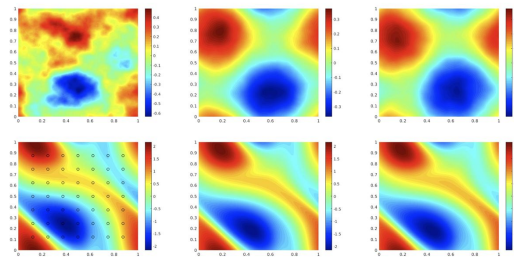


01

## Traditional PDE solvers:

Require fine meshes and solve one scenario at a time, computationally expensive for many scenarios

## Example 5: Bayesian inverse problem:



We use a MCMC method, sampling initial conditions and evaluating them with the traditional solver and Fourier operator. The Fourier operator takes **0.005s** to evaluate each initial condition, while the traditional solver takes **2.2s**.

Li, et al. 2021. Caltech and Purdue University

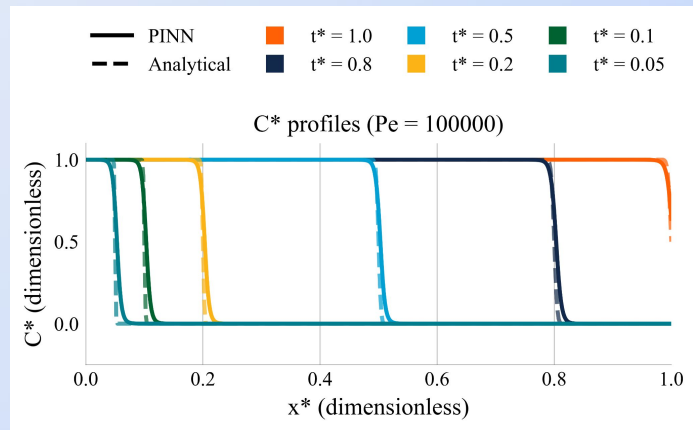
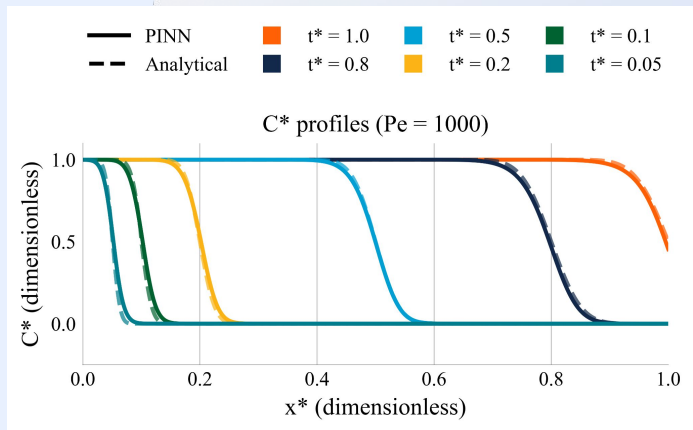
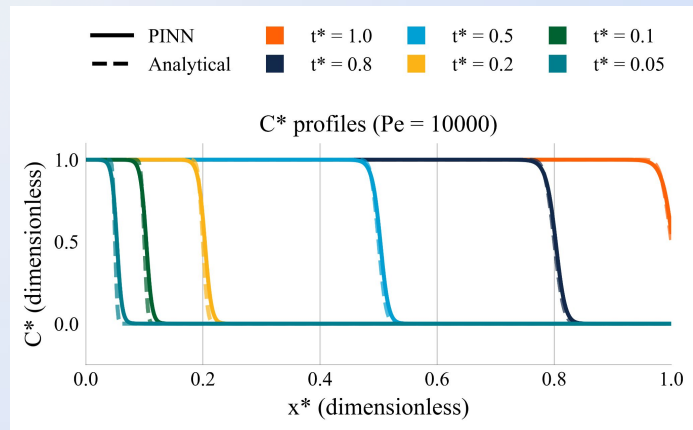
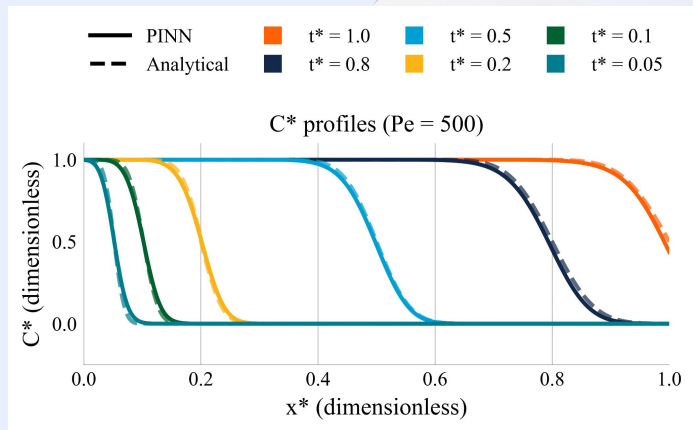
The training process, conducted on a single Nvidia Tesla A100 GPU, takes 6.5 hours over 200 epochs for each network. In comparison, a full PFLOTRAN forward simulation required approximately **1.5 hours** with one CPU. However, the U-FNO surrogate model predictions are computed in just **0.015 seconds**, resulting in a computational speedup of around 360,000 times.

Jiang, et al. 2024. Lawrence Berkeley international laboratory

02

## Physics-Informed Neural Networks (PINNs):

Train one neural net per scenario by embedding the PDE into the loss, avoids meshing but still **problem-specific** and can be slow to train for each new case.



03

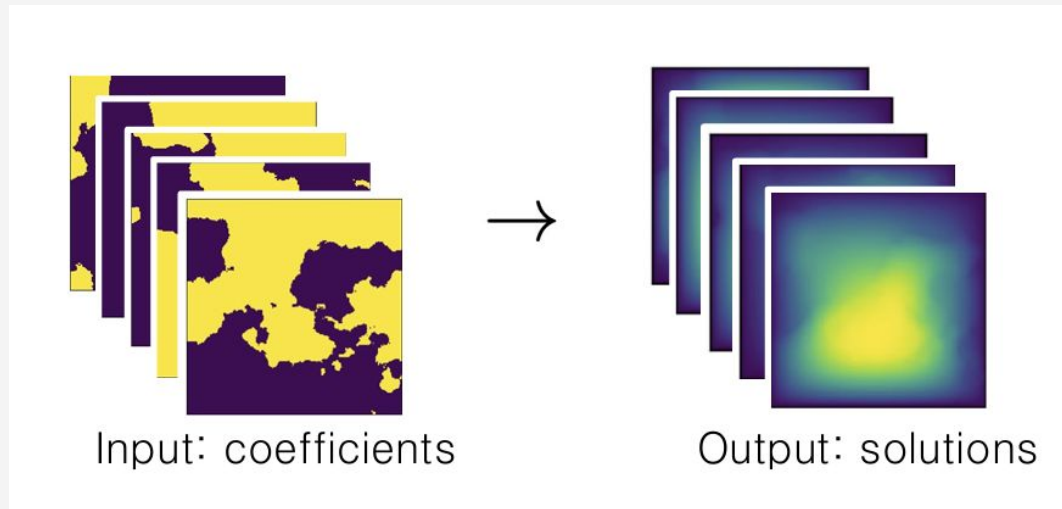
### Neural Operator Promise:

instead of solving a solution for one setup, they learn the solution map itself.

# Solve vs Learn

# What is a Neural Operator?

**Learn an operator  $G: X \rightarrow Y$  mapping input functions to output functions (for example, initial contaminant distribution to a future concentration field).**



**Key idea: represent function & operator in mesh-invariant way**

**Loss = Data Loss**

# Neural Operator Framework

An operator is a map between function spaces:

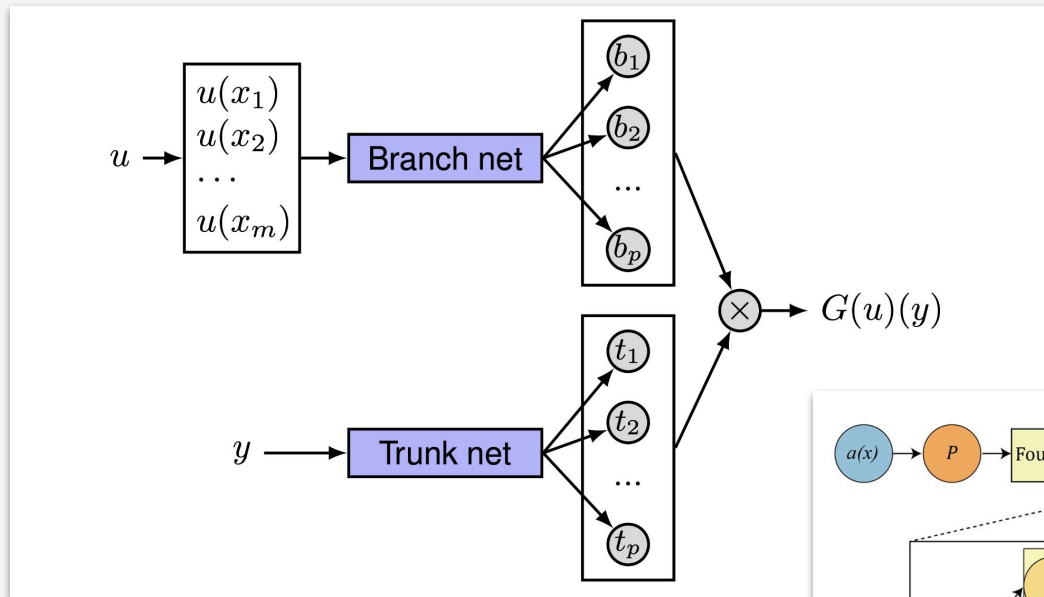
$u$ : an input function (e.g., coefficient field, initial condition, boundary forcing).

$v$ : an output function (e.g., PDE solution field at time  $T$ , or full space-time solution).

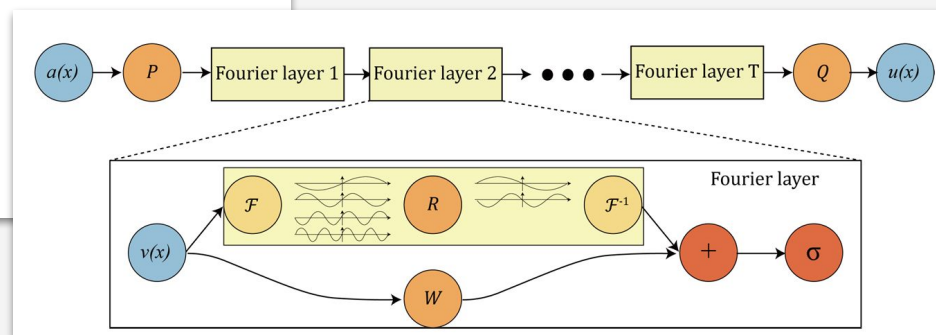
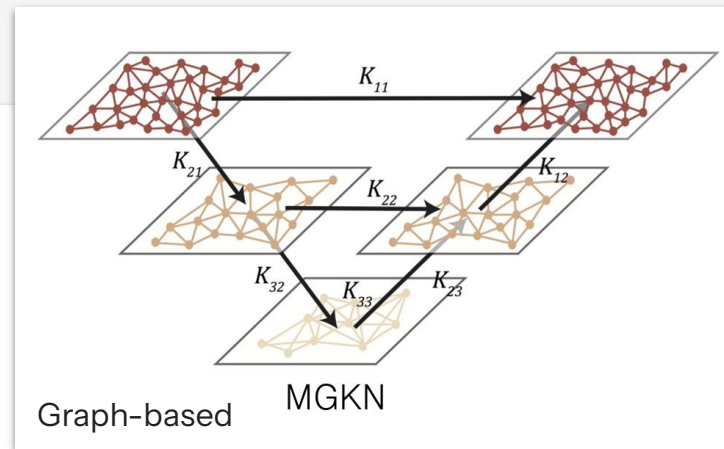
$$\mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}, \quad \underbrace{u(\cdot)}_{\text{Input function}} \mapsto \underbrace{v(\cdot)}_{\text{Output function}} = \underbrace{\mathcal{G}(u)}_{\text{Operator}}(\cdot)$$

PINN learns one solution for one PDE instance; a neural operator learns the PDE solver itself, mapping problem setups (fields/IC/BC) to solution fields, so new scenarios run instantly without retraining.

# Neural Operator Architectures



DeepONet



Fourier Based

# Key Takeaways and Review

- Learn the solver, not one solution: approximate the operator  $G: u \rightarrow v$  (setup  $\rightarrow$  solution field)
- Train once, reuse forever: expensive offline training, then fast inference for new cases.
- Neural operator  $\neq$  PINN by default: it's an operator-learning architecture trained primarily with data loss (supervised on solution fields).
- Physics-informed variant: PDE/IC/BC residual losses augment the data loss (often termed PINO / physics-informed neural operator)

$$\mathcal{G} : \mathcal{X} \rightarrow \mathcal{Y}, \quad \underbrace{u(\cdot)}_{\text{Input function}} \mapsto \underbrace{v(\cdot)}_{\text{Output function}} = \underbrace{\mathcal{G}(u)}_{\text{Operator}}(\cdot)$$

# DeepONet Theory

## Understanding *the operator architecture*

### DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators

Lu Lu<sup>1</sup>, Pengzhan Jin<sup>2</sup>, and George Em Karniadakis<sup>1</sup>

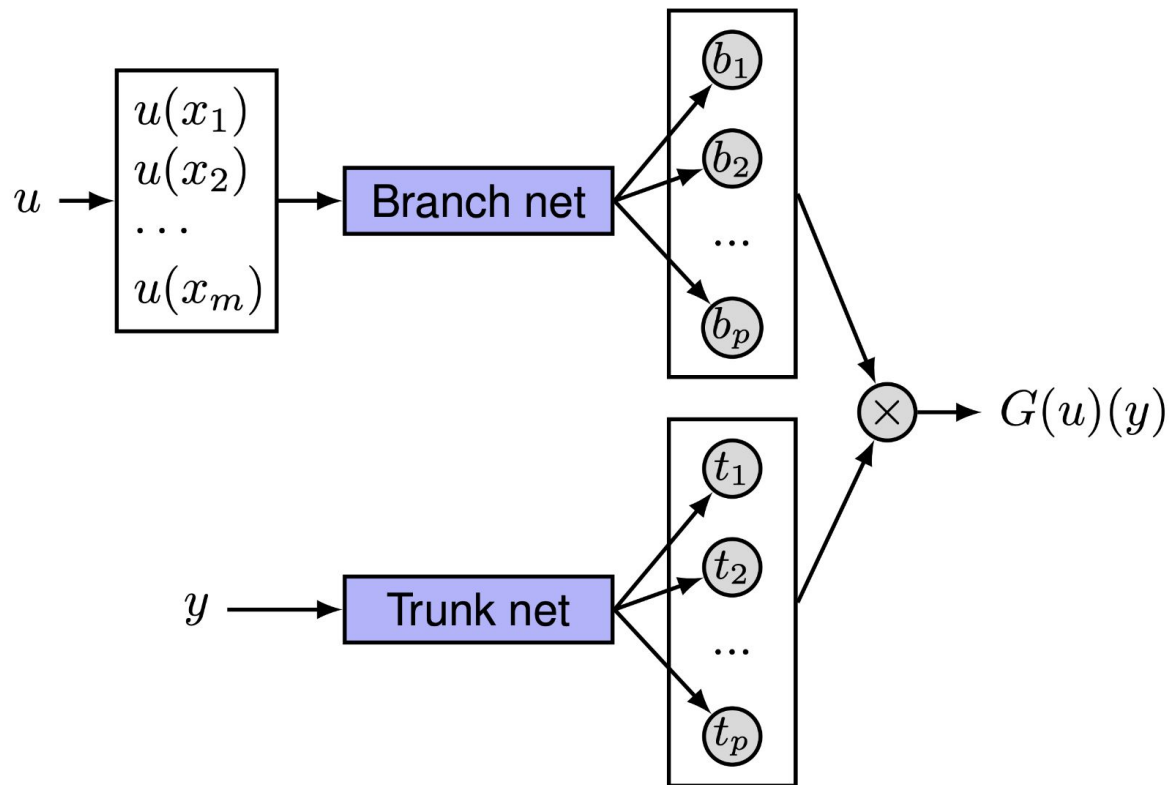
<sup>1</sup>Division of Applied Mathematics, Brown University, Providence, RI 02912, USA

<sup>2</sup>LSEC, ICMSEC, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China

#### Abstract

While it is widely known that neural networks are universal approximators of continuous functions, a less known and perhaps more powerful result is that a neural network with a single hidden layer can approximate accurately any nonlinear continuous operator [5]. This universal approximation theorem is suggestive of the potential application of neural networks in learning nonlinear operators from data. However, the theorem guarantees only a small approximation error for a sufficient large network, and does not consider the important optimization and generalization errors. To realize this theorem in practice, we propose deep operator networks (DeepONets) to learn operators accurately and efficiently from a relatively small dataset. A DeepONet consists of two sub-networks, one for encoding the input function at a fixed number of sensors  $x_i, i = 1, \dots, m$  (branch net), and another for encoding the locations for the output functions (trunk net). We perform systematic simulations for identifying two types of operators, i.e., dynamic systems and partial differential equations, and demonstrate that DeepONet significantly reduces the generalization error compared to the fully-connected networks. We also derive theoretically the dependence of the approximation error in terms of the number of sensors (where the input function is defined) as well as the input function type, and we verify the theorem with computational results. More importantly, we observe high-order error convergence in our computational tests, namely polynomial rates (from half order to fourth order) and even exponential convergence with respect to the training dataset size.





## Sensors and queries

$$\mathbf{u}_i = \begin{bmatrix} u_i(x_1) \\ \vdots \\ u_i(x_m) \end{bmatrix} \in \mathbb{R}^m.$$

Input sampling (sensors)

$$\mathbf{v}_i = \begin{bmatrix} v_i(y_1) \\ \vdots \\ v_i(y_q) \end{bmatrix} \in \mathbb{R}^q.$$

Output evaluation (queries)

$$\hat{\mathcal{G}}_{\theta}(u)(y) = \sum_{r=1}^p B_{\theta_B}^{(r)}(\mathbf{u}) T_{\theta_T}^{(r)}(y)$$

**Branch net**  $B(u)$ : takes samples of the input function (sensor values) and outputs **coefficients/features**.

**Trunk net**  $T(y)$ : takes the **query coordinate** and outputs a **basis vector**.

**Dot product / sum**: combines them into a scalar value at that coordinate  $y$ .

## Realistic example — heterogeneous problem

$$\partial_t C(x, t) + \underbrace{U(x)}_{\text{a function of } x} \partial_x C(x, t) = D \partial_{xx} C(x, t), \quad x \in [0, 1], \quad t \in [0, T].$$

a function of  $x$

$$C(x, 0) = C_0(x), \quad C(0, t) = C_{\text{in}}(t), \quad C(1, t) = 0.$$

IC: initial plume

IC: inlet concentration

$$\mathcal{G} : (U(\cdot), C_0(\cdot), C_{\text{in}}(\cdot)) \mapsto C(\cdot, \cdot).$$

$$\{x_i\}_{i=1}^m \subset [0, 1], \quad \{t_j\}_{j=1}^n \subset [0, T].$$

Sensor Sets

$$s(u) = \left[ U(x_1), \dots, U(x_m), C_0(x_1), \dots, C_0(x_m), C_{\text{in}}(t_1), \dots, C_{\text{in}}(t_n) \right] \in \mathbb{R}^{2m+n}.$$

Sensor Vector

$$y = (x, t) \in [0, 1] \times [0, T].$$

Query Point

$$b(u) = B_{\theta}(s(u)) \in \mathbb{R}^p.$$

Branch net

$$\tau(y) = T_{\phi}(y) \in \mathbb{R}^p, \quad y = (x, t).$$

Trunk net

$$\hat{C}(x, t) = \langle b(u), \tau(x, t) \rangle = \sum_{r=1}^p b_r(u) \tau_r(x, t).$$

Prediction (dot product)

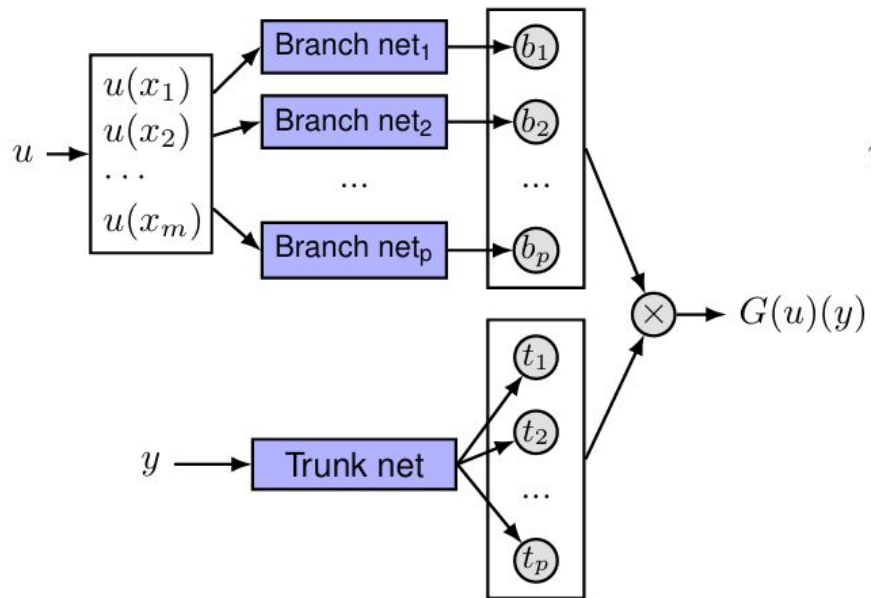
$p$  is the size of the shared latent feature vector: the branch outputs ( $p$ ) coefficients and the trunk outputs ( $p$ ) basis values so their dot product produces the prediction.

In training diverse functions of  $\mathcal{U}$  is used to make the model generalizable

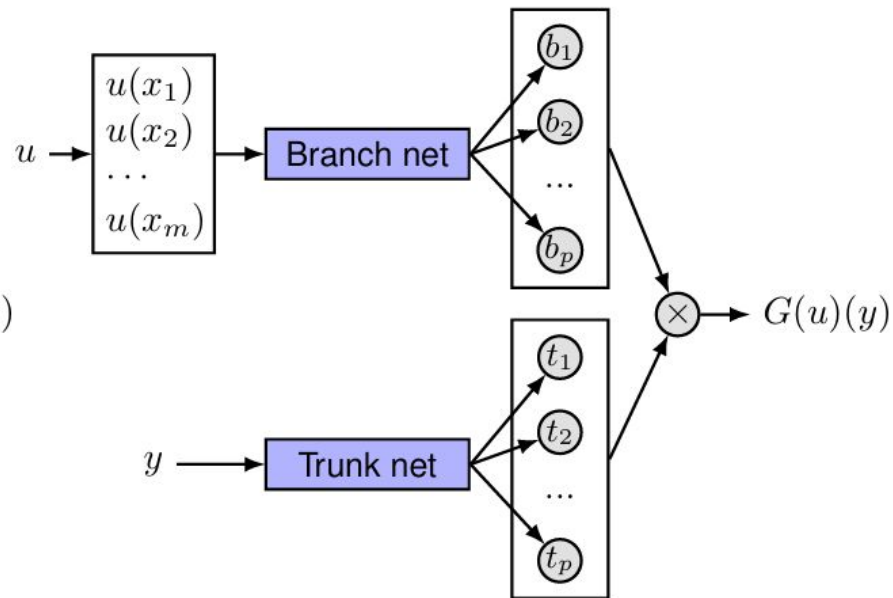
$$u^{(k)} = (U^{(k)}(\cdot), C_0^{(k)}(\cdot), C_{\text{in}}^{(k)}(\cdot)), \quad C^{(k)}(\cdot, \cdot) = \mathcal{G}(u^{(k)}), \quad k = 1, \dots, N.$$

$k$  is the indice of input functions used for training

**C** Stacked DeepONet



**D** Unstacked DeepONet

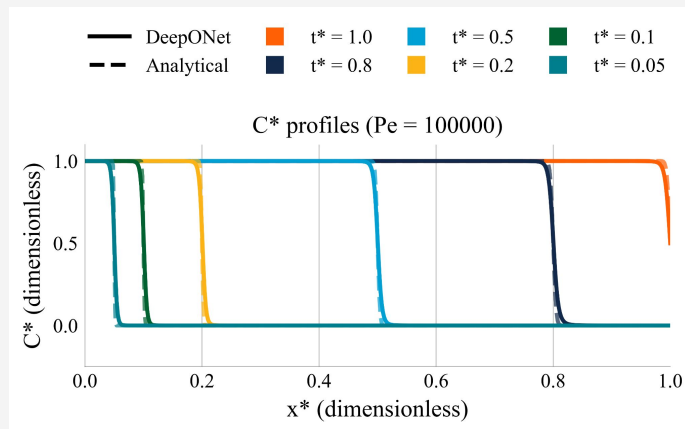
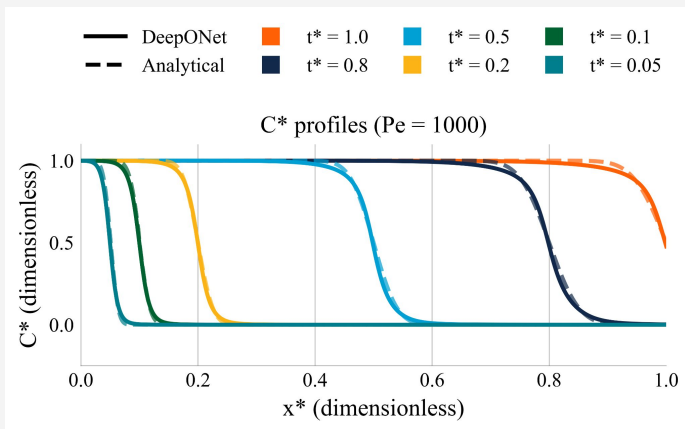
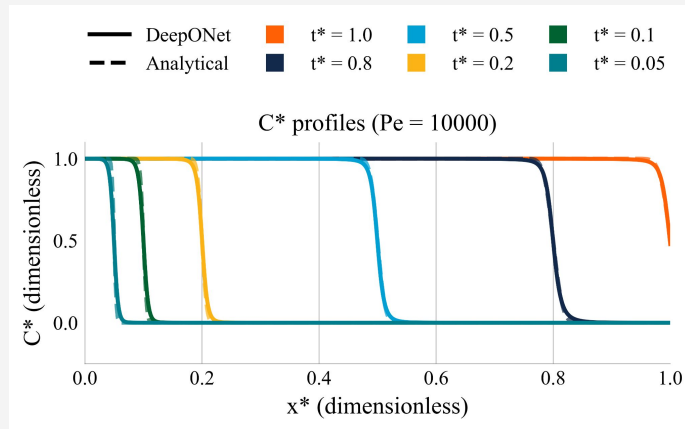
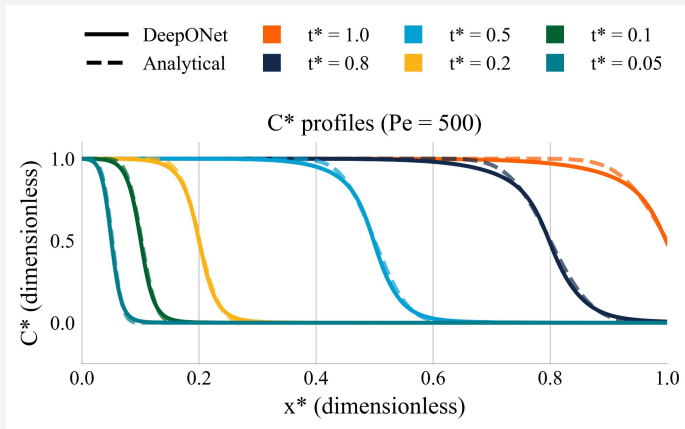




03

## Implementation in our problem





05

## Research Gap & Next Steps



# Key Studies Using Neural Operators for Groundwater Transport (Advection–Diffusion/Dispersion)

Huang et al., 2024  
Closest paper to our problem

Porous-DeepONet: Learning the Solution Operators of Parametric Reactive Transport Equations in Porous Media

[link](#)

Diab et al., 2024  
1D Two-phase flow  
(Buckley–Leverett)

Learning Generic Solutions for Multiphase Transport in Porous Media via the Flux Functions Operator

[link](#)

Kamil et al., 2025  
Richards equation

Physics-Informed Neural Operators for Efficient Modeling of Infiltration in Porous Media

[link](#)

Praditia et al., 2022  
Diffusion–sorption equation

Learning Groundwater Contaminant Diffusion–Sorption Processes with a Finite Volume Neural Network

[link](#)

Al Hashim et al., 2025  
Two-phase subsurface flow

A Dual-Branch Coupled Fourier Neural Operator for High-Resolution Multi-Phase Flow Modeling in Porous Media

[link](#)

# Research Gap

Despite the growing body of work applying neural operators to subsurface flow and transport, **no publication to date** explicitly focuses on the classical advection–diffusion/dispersion equation in a simple 1D groundwater setting.

# Next Steps

- |    |                                                                           |           |
|----|---------------------------------------------------------------------------|-----------|
| 00 | <b>Refine the Neural Operator implementation</b>                          | Next Week |
| 01 | <b>Advance to heterogeneous (more realistic) problem with many inputs</b> | Next Week |
| 02 | <b>Assess performance and efficiency of neural operators</b>              | Next Week |

# References

Lu et al., 2019

DeepONet **Founding Paper** ✨ ✨ ✨

DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators

[link](#)

Li et al., 2021

Slide Deck

Neural Operator For Parametric PDEs

[link](#)

Brunton, 2024

Helpful video teaching  
DeepONet

Deep Operator Networks (DeepONet)

[link](#)

Huang et al., 2024

Closest paper to our problem

Porous-DeepONet: Learning the Solution Operators of Parametric Reactive Transport Equations in Porous Media

[link](#)

# Thank you



Slidedeck available at [kutt.it/neuop](https://kutt.it/neuop)

